UC Number	UC Title	Description	Actor(s)	Methods/Understanding	Potential Implementations
REQ-001: Data Ingestion					
UC-Ingest-01	Schedule & Trigger Alpha Vantage News Fetch	Cron-triggered Lambda fetches financial news articles and sentiment scores from Alpha Vantage API; writes raw JSON to object storage.	Scheduler (System)	This service uses cron-based triggers to manage intermittent API polling, balancing the need for timely market updates with the constraints of strict API rate limits while ensuring raw payload integrity for downstream processes.	use Cloud Sheduler with cron to push messages to topics, which triggers Cloud Function/Cloud Run in Python which will fetch and securely write raw JSON to Cloud Storage bucket.
UC-Ingest-02	Ingest & Stage Raw News Article Records	Reads raw Alpha Vantage JSON from object storage, deduplicates by URL hash, and inserts new article records to DB with status=STAGED.	Ingest Worker (System)	This architecture implements a robust ingestion guardrail that processes raw object storage events, extracts nested news entries, and applies hash-based uniqueness checks to prevent duplicate database records during high-volume events.	Use Eventarc trigger or Cloud storage triggers for automatic notification when JSON object lands, which will initiate Cloud Run container that processes the file. Computes SHA256 hash of URL and writes deduplicated entries to Cloud SQL or Cloud Spanner
UC-Ingest-03	Schedule & Trigger Reddit Community Post Fetch	Cron-triggered Lambda fetches community posts from r/wallstreetbets, r/stocks, and r/investing via Reddit OAuth2 API (PRAW); writes raw JSON to object storage.	Scheduler (System)	By managing OAuth2 token lifecycles and periodic scraping, this mechanism efficiently targets high-velocity forums while addressing the inherent volatility of social media APIs and potential serverless execution time limits.	use Cloud Scheduler for periodic pings to Cloud Pubs/Sub topic for social scraping, use Cloud Run with PRAW to securely retrieve API secrets from Secret Manager and pull latest submissions, uploads raw multipost JSON to Cloud Storage bucket.
UC-Ingest-04	Ingest & Stage Raw Reddit Post Records	Reads raw Reddit JSON from object storage, deduplicates by native Reddit post ID, and inserts new post records to DB with status=STAGED.	Ingest Worker (System)	This process parses unstructured social payloads by leveraging atomic database operations to reconcile incoming posts with existing data, although high concurrency during deduplication requires careful management of transaction pools.	Set up a Cloud Run that is triggered by new file uploads in Reddit data bucket with Eventarc, which will stream JSON array, extract metadata, and utilize on conflict/do nothing SQL statement.
UC-Ingest-05	Schedule & Trigger SEC EDGAR Filing Fetch	Cron-triggered daily Lambda fetches 10-K, 10-Q, and 8-K filings from SEC EDGAR (no API key required); writes raw filing text to object storage.	Scheduler (System)	Reliable ingestion here hinges on maintaining proper User-Agent headers to adhere to SEC access policies while streaming large volumes of unstructured text data into manageable object storage buckets.	Create cron schedule with Cloud Scheduler, launch Cloud Run Job with User-Agent header for SEC RSS feedand downlod giles to stream to Cloud Storage
UC-Ingest-06	Ingest & Stage Raw EDGAR Filing Records	Parses raw filing HTML/XBRL, extracts section boundaries and structured XBRL financial metrics, and stages sec_filings DB record with section_map_json.	Ingest Worker (System)	This component performs resource-intensive document parsing to transform dense, inconsistent HTML into structured JSON, requiring careful memory allocation to handle large SEC filing files without triggering execution failures.	Trigger pipeline within Cloud Dataflow or async Cloud Run instance with expanded memory limits, code executrs parsing logic with BeautifulSoup or sec-api and records with JSONB data type
REQ-002: Data Pipelines					
UC-Pipe-01	[Pipeline A] Parse & Clean News Article Text	Strips residual HTML, normalizes whitespace, removes promotional boilerplate, and enforces 2,000-token budget on Alpha Vantage article summaries.	News Pipeline Worker (System)	Sanitization logic reduces noise in the input data by removing non-essential markers, yet overly aggressive filtering risks stripping subtle contextual cues that might be relevant for subsequent sentiment analysis.	Use a Cloud Run job running a Python script with the BeautifulSoup library to sanitize content.
UC-Pipe-02	[Pipeline A] Vectorize & Index News Article	Sends clean article text to embedding model (text-embedding-3-small); stores 1536-dim vector in vector DB with ticker and source metadata.	News Pipeline Worker (System)	This translation from text to multi-dimensional vectors allows for advanced semantic similarity searches, though indexing high-frequency updates requires batching strategies to prevent database write bottlenecks.	Use Vertex AI Embeddings API and store the results in Cloud SQL using the pgvector extension.
UC-Pipe-03	[Pipeline B] Parse & Clean Reddit Post Text	Strips Reddit markdown, removes bare URLs and bot signatures, normalizes whitespace, and applies low-signal quality flag for posts with score < 5 and comments < 3.	Community Pipeline Worker (System)	By applying heuristic-based filters, this system effectively separates meaningful investor discussion from low-signal noise, provided that the engagement thresholds remain calibrated to differentiate between genuine trends and spam.	Implement a filtering logic in a Cloud Function that reads raw JSON from Cloud Storage and writes clean text to a database.
UC-Pipe-04	[Pipeline B] Vectorize & Index Community Post	Sends clean Reddit post text to embedding model; stores vector with engagement metadata (score, num_comments, low_signal_flag) for post-retrieval filtering.	Community Pipeline Worker (System)	Encoding social content as vectors permits nuanced sentiment matching, though the slang-heavy nature of Reddit posts necessitates models capable of interpreting informal financial jargon.	Use Vertex AI for embeddings and store engagement metadata as JSON columns for efficient filtering.
UC-Pipe-05	[Pipeline C] Extract & Normalize EDGAR Filing Data	Parses 10-K/10-Q HTML into named sections (Risk Factors, MD&A, Financials); extracts XBRL financial metrics; stores section_map_json and filing_financials_json.	Filing Pipeline Worker (System)	Standardizing the highly variable structure of SEC filings requires sophisticated parsing that can map semi-structured HTML to a rigid schema, with the main hurdle being the inconsistent formatting between different filers.	Utilize Cloud Run with a containerized environment using libraries like sec-api to structure the XBRL/HTML data.
UC-Pipe-06	[Pipeline C] Vectorize & Index Filing Sections	Chunks each SEC filing section into 500-token overlapping windows; vectorizes each chunk; stores with section_name metadata to enable targeted RAG retrieval.	Filing Pipeline Worker (System)	Sliding window chunking ensures that specific sections of dense financial reports remain discoverable, though managing overlap is critical to avoid segmenting information in ways that break semantic coherence.	Apply a text-splitting strategy using LangChain inside a Cloud Run container before pushing chunks to the vector database.
UC-Pipe-07	[Shared] Execute RAG Inference & Prompt Assembly	Retrieves top-K relevant chunks from all three vector indexes, assembles a versioned source-labeled prompt, submits to LLM, and stores structured JSON result to inference_results.	Inference Engine (System)	The inference engine acts as the central intelligence by grounding model outputs in verified retrieved context, minimizing hallucination risks by strictly constraining the model to the provided documentation.	Use Vertex AI Gemini API with a system prompt that enforces source attribution to prevent hallucination.
UC-Pipe-08	[Shared] Generate Hype vs. Evidence Score	Computes weighted hype score (Reddit volume, speculative language, unsupported claims) and evidence score (XBRL financials, SEC citations, quantitative data); stores to ticker_scores.	Scoring Engine (System)	Quantifying the tension between speculation and hard data requires a weighted multi-factor model, yet these calculations remain sensitive to market volatility and subjective weighting assumptions.	Create a Cloud Function that runs periodic calculations comparing weighted metadata scores and updates the database.
UC-Pipe-09	[Shared] Persist Narrative State Update	Appends new snapshot to append-only narrative_state table; detects composite score shifts >= 10 points and dispatches alerts to subscribed users.	State Manager (System)	This mechanism creates an auditable historical record of narrative changes, but the storage demand for a high-frequency append-only log necessitates careful partitioning strategy to maintain query performance over time.	Utilize Cloud SQL partitioned tables and Cloud Pub/Sub to manage history and alert on significant movements.
REQ-003: Data Mapping					
UC-Map-01	Map News Article Entity to Ticker Symbol	Applies NER and fuzzy matching to resolve article text mentions to a ticker symbol; assigns confidence score; flags low-confidence records for admin review.	Mapping Worker (System)	Natural Language Processing techniques extract entity names which are then normalized via fuzzy string matching to ticker symbols, serving as a critical bridge between unstructured news and structured financial data.	Build a Cloud Run service using SpaCy for NER and a fuzzy string matching library to compare extracted names against a known ticker database.
UC-Map-02	Map EDGAR Filing to Ticker via CIK Lookup	Resolves SEC CIK number to ticker symbol via direct O(1) lookup in cik_ticker_mapping table; confidence always 1.0 for matched filings.	Mapping Worker (System)	This mapping utilizes the SEC's unique identifier (CIK) to establish a deterministic link to a ticker, ensuring perfect accuracy for official corporate financial filings.	Implement a look-up table within Cloud SQL that maps CIKs to Tickers and execute a standard SQL join operation.
UC-Map-03	Map Reddit Post to Ticker via Mention Detection	Scans post text for cashtag patterns and company name mentions; maps one post to one or more tickers via post_ticker_mapping join table.	Mapping Worker (System)	The system scans for specific financial syntax like '$TICKER' and common company naming conventions to associate informal social discussions with formal financial entities.	Create a Cloud Function that uses regex patterns to extract potential tickers from post text and saves the associations to a mapping table.

UC Number	UC Title	Description	Actor(s)	Methods/Understanding	Potential Implementations
UC-Map-04	Validate & Audit Mapping Confidence	Admin reviews MAPPING_REVIEW queue for all three content types; automated job tracks auto-map rate and alerts if accuracy falls below 80%.	Admin User + System	By establishing a human-in-the-loop review process, the system monitors the automated mapping accuracy and provides an escalation path when machine confidence levels are insufficient.	Develop a web dashboard using App Engine or Cloud Run for admins to review queued mapping records and update them in the database.
UC-Map-05	Flag and Log Mapping Error (User-Reported)	Authenticated user reports an incorrect ticker mapping from the dashboard UI; report is logged and queued for admin review — no immediate change to the record.	Authenticated User	User-provided feedback acts as a crowdsourced validation layer that improves system accuracy over time by surfacing mapping discrepancies that automated logic might miss.	Expose an API endpoint that creates a report in a dedicated 'errors' table in Cloud SQL to notify admins.
REQ-004: User Creation					
UC-User-01	Register Account	User creates a new account with email and password; includes email validation, bcrypt password hashing, and confirmation email.	Unauthenticated User	Security protocols during registration ensure that credentials are never stored in plain text and identity is verified via email, which is fundamental to maintaining user trust and platform integrity.	Use Firebase Authentication for managed user identity or a Cloud Function to handle custom registration logic with stored password hashes.
UC-User-02	Recover Account	User resets a forgotten password via a single-use, time-limited token delivered by email.	Unauthenticated User	By leveraging ephemeral tokens, this process facilitates secure account recovery while protecting against unauthorized access or brute-force attempts on sensitive user accounts.	Deploy a Cloud Function to generate a secure, time-limited token stored in Cloud SQL and send an email via SendGrid or similar service.
REQ-005: Authentication					
UC-Auth-01	Login & Credential Validation	User submits credentials; system validates against stored hash and issues a signed JWT session token.	Registered User	Authentication relies on standard cryptographic verification where the system compares hashes to confirm identity and establishes secure session state via signed tokens.	Implement an authentication middleware in a Cloud Run service that validates user credentials and issues a JWT token upon success.
UC-Auth-02	Session Lifecycle Management	System enforces sliding inactivity timeout, handles explicit logout, and rotates refresh tokens on renewal.	System (passive enforcement)	Maintaining session security through active token management and inactivity tracking ensures that stale sessions are pruned and users are consistently authorized, enhancing the platform's posture against session hijacking.	Use Redis to store session tokens and implement a background worker or middleware to manage token rotation and expiration.
REQ-006: Search					
UC-Search-01	Execute User Keyword Search	User enters ticker symbol, company name, or topic keyword; system returns matched tickers with current hype/evidence scores and narrative summaries.	Authenticated User	This search layer synthesizes structured data (scores) with unstructured data (narratives), allowing users to immediately view the health and sentiment surrounding a financial entity.	Build a search API on Cloud Run that queries the Cloud SQL database for ticker-specific scores and recent narrative summaries.
UC-Search-02	Rank & Return Search Results	Internal ranking engine applies weighted scoring (semantic similarity, recency, data completeness) to candidate results from DB lookup and vector search.	Search Ranking Engine (System)	A multi-factor ranking algorithm balances relevance by combining vector-based semantic matches with traditional metrics like recency, ensuring the most valuable information is surfaced first.	Develop a scoring service that combines vector search results with metadata scores using weighted arithmetic in a Cloud Run instance.
UC-Search-03	Filter Search Results by Date Range	User applies start/end date filter to view articles, posts, filings, and score history for a selected ticker within the specified window.	Authenticated User	Date filtering allows users to isolate data points to specific time frames, enabling time-series analysis that is crucial for financial tracking.	Add 'created_at' index to the SQL tables and include a date range filter in the query parameters for search APIs.
UC-Search-04	Execute Semantic Vector Search (App-Internal RAG)	Generates query embedding and executes cosine similarity search against all three pipeline indexes with optional metadata filters (ticker, source type, date, section).	Inference Engine (System)	Semantic search uses vector math to find documents that are conceptually similar to a user's query, providing a superior retrieval experience compared to literal keyword matching.	Utilize the pgvector extension's cosine similarity function in Cloud SQL to perform vector searches based on the input query embedding.
UC-Search-05	Augment LLM Prompt with Retrieved Context	Assembles versioned RAG prompt from source-labeled context chunks; enforces 75% token budget; submits to LLM; returns structured JSON output.	Inference Engine (System)	Context augmentation constructs a precise prompt for the LLM by combining user queries with top-retrieved documents, ensuring the response is both accurate and grounded in the latest evidence.	Use a prompt template manager that iterates over retrieved document chunks and truncates them to fit the token budget before sending to the LLM.
UC-Search-06	Decompose & Execute Multi-Hop Query	Detects multi-ticker or topic-wide query intent; executes parallel sub-searches per ticker; synthesizes comparative LLM output with per-ticker attribution.	Query Router (System)	Multi-hop decomposition breaks complex broad questions into smaller, executable sub-tasks, allowing the system to aggregate information across diverse tickers or topics before presenting a holistic answer.	Create a query router function that uses a lightweight LLM call to identify sub-queries and orchestrate parallel execution using Pub/Sub.
UC-Search-07	Cache & Return Repeated Search and Inference Results	Redis cache intercepts all vector search and LLM inference calls; returns result on cache hit; populates cache with source-type TTL (15–60 min) on miss.	Cache Layer (System)	Implementing a caching layer dramatically reduces redundant LLM compute costs and latency by storing and reusing previous inference results for frequently accessed topics.	Deploy a Memorystore for Redis instance to store key-value pairs of query embeddings and their resulting answers.
REQ-007: Workspace					
UC-Work-01	Create & Manage Research Workspace	User creates a named workspace to group tickers for ongoing research; supports rename and soft-delete with data archival (no hard delete of historical ticker data).	Authenticated User	Workspaces offer a persistent container for user research, requiring careful database schema design to maintain associations between users and their chosen ticker lists while ensuring archival integrity.	Store workspace definitions and ticker associations in relational tables in Cloud SQL, using a boolean flag for soft-deletes.
UC-Work-02	Add & Remove Ticker from Workspace	User adds or removes tickers from a workspace; system enforces hard 50-ticker limit and triggers cache invalidation on removal.	Authenticated User	Managing workspace contents requires enforcing business logic limits while ensuring that state updates are immediately reflected across the system to maintain data consistency for the user.	Implement an API endpoint that validates the workspace ticker count and issues a cache invalidation signal to the Redis layer upon update.
REQ-008: Audit Trail					
UC-Audit-01	Source Link Management & Integrity Validation	Stores source URL with every article, post, and filing record at ingest; daily job validates all URLs; broken links flagged in DB and displayed as 'Source Unavailable' in UI.	System (automated) + Authenticated User	Maintaining source integrity involves storing original URLs at the point of ingestion and running background validation jobs, which serves to preserve the provenance of the research.	Create a scheduled Cloud Run Job that probes stored URLs periodically and updates the status of the record in the database.
REQ-009: Timeline					
UC-Time-01	Render & Navigate Narrative Timeline	Displays chronological event history from the append-only narrative_state table; lazy-loads older events on scroll; event markers link to source documents.	Authenticated User	Timeline rendering leverages append-only data patterns for chronological display, and the use of lazy-loading ensures smooth performance even when accessing extensive historical datasets.	Implement an API that supports pagination and sorting on the narrative_state table in Cloud SQL to serve data as the user scrolls.
UC-Time-02	Set Alerts & Export Timeline View	User subscribes to narrative shift alerts for a ticker; exports current timeline view as a downloadable PDF or CSV including source citation URLs.	Authenticated User	Subscription systems require asynchronous event monitoring, while exporting data to portable formats requires a data transformation service to ensure the output remains readable and cited.	Use Cloud Pub/Sub for alert notifications and a Cloud Function that executes a SQL query to fetch timeline records for CSV/PDF generation.
REQ-010: Dashboard					

UC Number	UC Title	Description	Actor(s)	Methods/Understanding	Potential Implementations
UC-Dash-01	Render Ticker Research Dashboard	Authenticated user opens a ticker page and views aggregated research: news summaries, hype/evidence scores, filing highlights, and community sentiment. Full load < 3 seconds.	Authenticated User	A high-performance dashboard requires efficient data fetching, often involving pre-aggregated materialized views in the database to ensure that complex research summaries load within the target timeframe.	Design materialized views in Cloud SQL for aggregated ticker data and serve these through a performant backend API on Cloud Run.